

RMSNorm & SwiGLU.

LLaMA swaps two of the transformer's standard parts. RMSNorm replaces LayerNorm by normalising with the root-mean-square only — no mean subtraction, no bias — which is cheaper and just as good. SwiGLU replaces the GELU feed-forward with a *gated* unit: $(\text{Swish}(xW_1) \odot xW_3)W_2$, whose elementwise gate buys expressivity, paid for by shrinking the hidden width to $\frac{2}{3} \cdot 4d$ so the parameter count stays put. We compute both on real vectors. Every number below is computed in full — nothing summarised or deferred.

THE EQUATIONS

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \varepsilon}} \odot g, \quad \text{FFN}(x) = (\text{Swish}(xW_1) \odot xW_3)W_2$$

Worked example. RMSNorm vs LayerNorm of $x = [2, -1, 0.5, 1.5]$; Swish at $z = -2, -1, 0, 1, 2$; a small vector pushed through a $2 \rightarrow 3 \rightarrow 2$ SwiGLU FFN.

Topic 3 of 3 in this module · companion to the course page · v1.0

Contents

1	RMSNorm: normalise by magnitude, not by mean	2
2	Worked example: RMSNorm vs LayerNorm of the same vector	3
3	SwiGLU: a gated feed-forward	4
4	Keeping parameters equal: the $\frac{2}{3} \cdot 4d$ hidden width	5
5	Properties / consequences that matter	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 RMSNorm: normalise by magnitude, not by mean

LayerNorm centres a vector (subtract the mean μ), scales by its standard deviation, then applies a learned gain g and bias β . RMSNorm keeps only the scaling step, using the root-mean-square in place of the standard deviation, and drops both the mean subtraction and the bias:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}} \odot g \quad (1)$$

Compare LayerNorm, which first centres: $\text{LN}(x) = \frac{x - \mu}{\sqrt{\frac{1}{d} \sum_i (x_i - \mu)^2 + \varepsilon}} \odot g + \beta$, with $\mu = \frac{1}{d} \sum_i x_i$.

Symbol	Meaning	Value / shape
x	layer input vector	$\in \mathbb{R}^d$
d	model dimension	here 4
g	learned per-dim gain	$\in \mathbb{R}^d$ (here 1)
ε	numerical floor	10^{-5} (negligible here)
$\text{RMS}(x)$	$\sqrt{\frac{1}{d} \sum x_i^2}$	scalar

LayerNorm answers “where is this vector relative to its own centre, in units of its spread?”. RMSNorm drops the question of *where the centre is* and only rescales by overall magnitude: “how big is this vector, root-mean-square?”. Empirically the re-centring contributes little

to why normalisation helps — the stabilising effect comes from controlling scale — so removing μ and β costs no quality while saving a mean, a subtraction, and a parameter vector per layer.

2 Worked example: RMSNorm vs LayerNorm of the same vector

Take $x = [2.0, -1.0, 0.5, 1.5]$, $d = 4$, $g = 1$, ε negligible.

2.1 RMSNorm

Sum of squares and mean square:

$$\sum_i x_i^2 = 2.0^2 + (-1.0)^2 + 0.5^2 + 1.5^2 = 4 + 1 + 0.25 + 2.25 = 7.5, \quad \frac{1}{d} \sum_i x_i^2 = \frac{7.5}{4} = 1.875.$$

$$\text{RMS}(x) = \sqrt{1.875} = 1.369306.$$

Divide each component by the RMS:

$$\text{RMSNorm}(x) = \frac{1}{1.369306} \begin{bmatrix} 2.0 \\ -1.0 \\ 0.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 1.460593 \\ -0.730297 \\ 0.365148 \\ 1.095445 \end{bmatrix}.$$

2.2 LayerNorm (centres first)

Mean, then variance about the mean:

$$\mu = \frac{1}{4}(2.0 - 1.0 + 0.5 + 1.5) = \frac{3.0}{4} = 0.75,$$

$$\begin{aligned} \frac{1}{d} \sum_i (x_i - \mu)^2 &= \frac{1}{4}(1.25^2 + (-1.75)^2 + (-0.25)^2 + 0.75^2) \\ &= \frac{1}{4}(1.5625 + 3.0625 + 0.0625 + 0.5625) = 1.3125, \end{aligned}$$

so the standard deviation is $\sqrt{1.3125} = 1.145644$. Centre and scale:

$$\text{LN}(x) = \frac{1}{1.145644} \begin{bmatrix} 2.0 - 0.75 \\ -1.0 - 0.75 \\ 0.5 - 0.75 \\ 1.5 - 0.75 \end{bmatrix} = \begin{bmatrix} 1.091089 \\ -1.527525 \\ -0.218218 \\ 0.654654 \end{bmatrix}.$$

The two differ markedly — e.g. the second component is -0.730 under RMSNorm but -1.528 under LayerNorm — because LayerNorm first shifts everything down by $\mu = 0.75$. Side by side (shading $\propto |\text{value}|$, capped at `indigo!80`):

	x_1	x_2	x_3	x_4
RMSNorm	1.4606	-0.7303	0.3651	1.0954
LayerNorm	1.0911	-1.5275	-0.2182	0.6547

RMSNorm = LayerNorm minus the mean-centring and minus the bias. It keeps the one thing that matters for stable training — rescaling by magnitude — and discards the rest. Same quality, fewer operations, one fewer parameter vector per normalisation. That is the whole reason LLaMA uses it.

3 SwiGLU: a gated feed-forward

The standard FFN is $\text{FFN}(x) = \phi(xW_1)W_2$ with $\phi = \text{GELU}$ and two matrices. SwiGLU splits the input projection into *two* projections and multiplies them elementwise, with one side passed through Swish as a gate:

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}(xW_1) \odot xW_3)W_2, \quad \text{Swish}(z) = z \sigma(\beta z) \quad (2)$$

With $\beta = 1$, $\text{Swish} = \text{SiLU}(z) = z \sigma(z)$, where $\sigma(z) = 1/(1 + e^{-z})$. The $\odot$ is an element-wise (Hadamard) product of the two hidden projections: xW_3 is a value branch, $\text{Swish}(xW_1)$ a multiplicative gate that can suppress or amplify it per coordinate.

3.1 Swish at $z = -2, -1, 0, 1, 2$

$$\begin{aligned} \text{Swish}(z) = z \sigma(z), \quad \sigma(-2) = 0.119203, \sigma(-1) = 0.268941, \sigma(0) = 0.5, \\ \sigma(1) = 0.731059, \sigma(2) = 0.880797. \end{aligned}$$

z	-2	-1	0	1	2
$\sigma(z)$	0.119203	0.268941	0.500000	0.731059	0.880797
$\text{Swish}(z)$	-0.238406	-0.268941	0.000000	0.731059	1.761594

Swish is smooth and *non-monotone*: it dips slightly negative for small negative z (minimum near $z \approx -1.28$) before rising, unlike ReLU’s hard zero — this small negative “leak” is part of why it trains well.

3.2 A $2 \rightarrow 3 \rightarrow 2$ SwiGLU pass

Let $x = [1, 2]$ and (toy) weights

$$W_1 = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix}, \quad W_3 = \begin{bmatrix} 0.5 & 1 & 0 \\ 1 & 0 & 2 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Gate pre-activation xW_1 and value branch xW_3 :

$$xW_1 = [1 \cdot 1 + 2 \cdot 0, 1 \cdot 0 + 2 \cdot 1, 1 \cdot (-1) + 2 \cdot 1] = [1, 2, 1],$$

$$xW_3 = [1 \cdot 0.5 + 2 \cdot 1, 1 \cdot 1 + 2 \cdot 0, 1 \cdot 0 + 2 \cdot 2] = [2.5, 1, 4].$$

Apply Swish to the gate ($\text{Swish}(1) = 0.731059$, $\text{Swish}(2) = 1.761594$):

$$\text{Swish}(xW_1) = [0.731059, 1.761594, 0.731059].$$

Elementwise product (the gating step):

$$\text{Swish}(xW_1) \odot xW_3 = [0.731059 \cdot 2.5, 1.761594 \cdot 1, 0.731059 \cdot 4] = [1.827646, 1.761594, 2.924234].$$

Project back down with W_2 :

$$\begin{aligned} \text{FFN}(x) &= [1.827646, 1.761594, 2.924234] \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \\ &= [1.827646 + 2.924234, 1.761594 + 2.924234] = [4.751881, 4.685828]. \end{aligned}$$

4 Keeping parameters equal: the $\frac{2}{3} \cdot 4d$ hidden width

A GELU-FFN has two matrices of size $d \times 4d$ and $4d \times d$: $8d^2$ parameters. SwiGLU has *three* matrices ($W_1, W_3 : d \times d_{ff}$ and $W_2 : d_{ff} \times d$), so to match the parameter budget $3d d_{ff} = 8d^2$ requires

$$d_{ff} = \frac{8}{3}d = \frac{2}{3} \cdot 4d.$$

For $d = 4096$: the GELU hidden width is $4d = 16384$; SwiGLU uses $d_{ff} = \frac{2}{3} \cdot 16384 \approx 10922$ (rounded to a hardware-friendly multiple in practice). Three narrower matrices, same total weights, strictly more expressive than the two wide ones.

5 Properties / consequences that matter

1. **RMSNorm is cheaper.** No mean, no centring subtraction, no β — one pass over the vector and one fewer parameter set per norm, with no measured quality loss.
2. **Scale invariance.** $\text{RMSNorm}(cx) = \text{RMSNorm}(x)$ for any $c > 0$ (the RMS scales by $|c|$ and cancels), so it stabilises activation magnitude exactly like LayerNorm.
3. **Gating adds expressivity.** The $\odot$ lets one projection modulate another per coordinate — a multiplicative interaction a single GELU-FFN cannot represent at the same width.
4. **Param-neutral by design.** The $\frac{2}{3} \cdot 4d$ rule keeps SwiGLU’s three matrices at the same total size as GELU’s two, so the upgrade is free in parameters.

Two easy mistakes. (1) RMSNorm has *no* bias β and does *no* mean subtraction — if you “port” a LayerNorm by only deleting β but still subtract μ , you have not built RMSNorm. (2) SwiGLU needs *three* weight matrices, not two; if you keep the GELU-FFN’s $d_{ff} = 4d$ for all three you inflate the FFN parameters by $1.5\times$. Always shrink to $d_{ff} \approx \frac{2}{3} \cdot 4d$ to stay parameter-matched.

6 Where this sits in the track

RMSNorm and SwiGLU are, with RoPE (Topic 1) and GQA (Topic 2), the four substitutions that define the LLaMA block relative to the original transformer of Module 0.1–0.2. RMSNorm

replaces the LayerNorm of Module 0.2; SwiGLU replaces that module's GELU-FFN while preserving its parameter count. Together they make a decoder that is cheaper per layer and trains stably at scale — the architecture the rest of Phase 3 builds on.

A Reproducible (NumPy)

Inputs: $x = [2, -1, 0.5, 1.5]$ for the norms; $z \in \{-2, -1, 0, 1, 2\}$ for Swish; the $2 \rightarrow 3 \rightarrow 2$ weights above. The snippet reproduces $\text{RMS} = 1.369306$, both normalised vectors, the Swish table, and the FFN output $[4.751881, 4.685828]$.

```
import numpy as np
x = np.array([2.0, -1.0, 0.5, 1.5]); d = len(x)
rms = np.sqrt(np.mean(x**2)); print(rms, x/rms)           # 1.369306 ; RMSNorm(x)
mu = x.mean(); std = x.std()                             # std = sqrt(mean((x-mu)^2))
print(mu, std, (x-mu)/std)                               # 0.75 1.145644 ; LayerNorm(x)
sig = lambda z: 1/(1+np.exp(-z)); swish = lambda z: z*sig(z)
print([round(swish(z),6) for z in [-2,-1,0,1,2]])
# [-0.238406, -0.268941, 0.0, 0.731059, 1.761594]
xv = np.array([1.0, 2.0])
W1 = np.array([[1,0,-1],[0,1,1]]); W3 = np.array([[0.5,1,0],[1,0,2]])
W2 = np.array([[1,0],[0,1],[1,1]])
out = (swish(xv@W1) * (xv@W3)) @ W2
print(out)                                               # [4.751881 4.685828]
print(int(2/3*4*4096))                                  # 10922 (= 2/3 * 4d, d=4096)
```